

STARWEAVER: MANUAL DE ARQUITECTURA TÉCNICA

Versión: 2.5 Ampliada y Actualizada

Descripción: Documentación Completa de Vuelo, Combate, Simulación Orbital y Análisis Arquitectónico.

Este documento proporciona un desglose exhaustivo y riguroso de la arquitectura del software de vuelo espacial desarrollado para el proyecto StarWeaver en el motor de desarrollo Unity. A continuación, se detalla la responsabilidad de cada componente, las ecuaciones físicas que rigen el entorno, un análisis crítico del estado actual de la arquitectura con sus respectivos puntos de mejora, errores activos y la estructura de organización de archivos recomendada para garantizar un sistema robusto, modular y altamente eficiente.

1. Análisis General de la Arquitectura

El proyecto StarWeaver cuenta con una base sólida y excelentes intenciones de diseño arquitectónico. Destacan la separación estricta de responsabilidades mediante la interfaz contractual `IShipDriver`, el uso inteligente de `ScriptableObjects` para la configuración modular de armamento (`WeaponStats`), y la implementación de un sistema desacoplado de eventos dentro de `HealthComponent`. No obstante, el sistema actual presenta problemas de duplicidad de responsabilidades, dependencias conflictivas con sistemas heredados (`Legacy Input System`), clases huérfanas y errores de ejecución activos (referencias nulas) que deben mitigarse para asegurar la escalabilidad del simulador.

2. Desglose de Componentes por Capas y Espacios de Nombres

Capa Núcleo Global (Namespace: `StarWeaver.Core`)

- **OrbitalStarshipController.cs (Capa: Motor Físico / Ejecutor de Fuerzas):** Representa los músculos y la lógica de locomoción de la nave espacial. Coordina de forma continua la física orbital, los sistemas de postcombustión (`Boost`), fuerzas G y las rotaciones angulares complejas. Toma el paquete de datos `ShipInputState` provisto por su driver asignado y traduce esos valores numéricos en vectores de fuerza reales aplicados sobre el componente `Rigidbody` nativo en el bucle físico sincronizado (`FixedUpdate`). Su lógica está completamente depurada de fricciones de fluidos o resistencias de aire lineales, permitiendo un comportamiento inercial puro del espacio exterior. Soporta dos modos de

operación automatizados:

- *Modo Físico*: Simulación de alta fidelidad con RigidBody real orientada al jugador y naves en situaciones de combate cercano.
- *Modo Cinemático (LERP)*: Interpolación lineal optimizada para naves controladas por la Inteligencia Artificial que se encuentran fuera del rango de renderizado visual.
- **OrbitalManager.cs (Capa: Físicas de Entorno / Núcleo Global)**: Es el núcleo absoluto de la simulación astronómica de N-cuerpos dentro del juego. Utilizando un patrón estructural centralizado (Singleton), recolecta de forma activa todos los cuerpos rígidos identificados bajo la etiqueta de datos "Celestial" y aplica de manera mutua la ecuación de Gravitación Universal de Newton en cada paso del bucle de física integrado (FixedUpdate). A diferencia de los enfoques convencionales de gravedad lineal presentes de forma nativa en Unity, calcula la magnitud vectorizada de la fuerza de atracción entre cada par de masas según la distancia relativa que los separa, permitiendo órbitas elípticas, parabólicas e hiperbólicas dinámicas reales basadas en la fórmula:

$$\mathbf{F} = \mathbf{G} \cdot (m_1 \cdot m_2) / r^2$$

Donde G es la constante gravitacional configurada, m_1 y m_2 corresponden a las masas de los respectivos objetos celestes y r representa la distancia de separación instantánea en el espacio tridimensional.

- **ShipProperties.cs (Capa: Configuración de Datos de Hardware)**: Centraliza todas las especificaciones mecánicas e inerciales que dictan el comportamiento y límites dinámicos de un chasis de nave específico. Define variables de ingeniería vitales como la masa total expresada en kilogramos (crucial para los cálculos del OrbitalManager), los torques angulares de rotación y las potencias de salida de los propulsores principales. Adicionalmente, almacena variables de velocidad convencional máxima, comportamiento térmico y de consumo energético del sistema de Supersalto (Hyperdrive) y el Turbo. Actualmente está estructurado como un MonoBehaviour, lo que representa una oportunidad de migración hacia ScriptableObject.
- **ShipInputState.cs (Capa: Contenedor de Datos / Estructura de Control)**: Estructura de datos (struct de valor) completamente inmutable por frame encargada de empaquetar de forma limpia y organizada todas las intenciones de control generadas por un piloto. Contiene valores normalizados de ejes tridimensionales como empuje longitudinal (Thrust), traslación en el eje vertical (Vertical), y rotaciones angulares complejas de cabeceo, guiñada y alabeo (Pitch, Yaw, Roll), además de banderas booleanas críticas para el disparo de armamento primario, secundario, bombas de caída libre y postcombustor.
- **IShipDriver.cs (Capa: Interfaz de Abstracción / Contrato de Piloto)**: Define la interfaz contractual estricta que debe implementar cualquier entidad encargada de controlar o

pilotar una nave espacial. Actúa como una capa intermedia que desacopla por completo la fuente de entrada de comandos de la mecánica interna de movimiento, permitiendo procesar exactamente de la misma manera las directivas provistas por un jugador humano, una IA estratégica o un sistema cinemático guiado por código.

- **OrbitalPlayerDriver.cs:** Driver específico del jugador encargado de consumir el componente centralizado InputProvider. Actualmente duplica responsabilidades funcionales con PlayerShipDriver.

Capa de Entrada de Datos (Namespace: StarWeaver.Input)

- **InputProvider.cs:** Singleton centralizado encargado de la gestión de entradas físicas. Utiliza el sistema de input heredado de Unity (Legacy Input System), lo cual genera un conflicto arquitectónico potencial si el nuevo sistema de Input (New Input System) se encuentra activo de forma exclusiva en los Player Settings del motor.

Capa del Jugador (Namespace: StarWeaver.Player)

- **PlayerShipDriver.cs (Capa: Controlador del Jugador Humano):** Implementa directamente el contrato de la interfaz IShipDriver. Su función primordial es monitorear el subsistema de hardware activo, capturar las señales físicas del usuario en tiempo real mediante UnityEngine.Input (Legacy) y mapearlas de manera precisa en la estructura ShipInputState. Adicionalmente, interactúa de forma directa con la cámara de juego activa para proyectar vectores espaciales hacia el infinito mediante Raycasting, determinando el punto tridimensional exacto del mundo al cual los cañones de la nave deben apuntar de forma dinámica. Presenta una duplicidad de código directa con OrbitalPlayerDriver.

Capa de Sistemas Independientes (Namespace: StarWeaver.Systems)

- **ShipWeaponController.cs (Capa: Administrador de Armamento):** Maneja de manera lógica el hardware de ataque de la nave. Controla los tiempos internos de recarga en base al cooldown de cadencia definido en WeaponStats y cicla de forma secuencial los disparos entre los distintos puntos de anclaje de cañones físicos (Muzzle Transforms) configurados en el chasis. Al instanciar un proyectil en el espacio, calcula de forma inercial la velocidad combinada, sumando vectorialmente la velocidad lineal actual de la nave a la velocidad de salida de la bala, garantizando una física balística espacial realista.
- **WeaponStats.cs (Capa: Contenedor de Configuración de Datos):** Plantilla basada en ScriptableObject que permite a los diseñadores crear y configurar perfiles balísticos y de armas de manera modular desde el editor de Unity. Almacena coeficientes de daño por

impacto, velocidades iniciales de eyección, rango operativo máximo antes de la autodestrucción y cadencia de fuego, vinculando además recursos gráficos y auditivos esenciales como el Prefab del proyectil, destellos de partículas (Muzzle Flash) y efectos de audio.

- **HealthComponent.cs (Capa: Simulador de Supervivencia y Vitalidad):** Administra los puntos vitales del casco estructural y la matriz energética de los escudos deflectores mediante el uso desacoplado de UnityEvents. Incluye un algoritmo automatizado de regeneración de escudos que se activa condicionalmente si la nave pasa un intervalo de tiempo específico (delay) sin recibir impactos hostiles. Está preparado nativamente para expandir su lógica hacia mecánicas complejas de supervivencia ambiental (monitoreo de degradación por radiación cósmica y niveles de consumo de oxígeno).
- **OrbitalVisualization.cs:** Componente encargado de dibujar en pantalla las órbitas y las esferas de influencia espacial (SOI) utilizando un componente LineRenderer. Actualmente cuenta con un error activo de referencia nula (NullReferenceException).

Capa de Automatización Cinemática y Control de Cámaras (Scripts Suelto / Raíz)

- **NaveSplineFollow.cs (Capa: Automatización Cinemática / Navegación por Guías):** Anula temporalmente la lectura de fuerzas físicas del controlador inercial para interpolar la posición y rotación de la nave espacial a lo largo de una curva o spline predefinido en la escena (Unity SplineContainer). Utilizado para automatizar secuencias introductorias, aproximaciones de acople a estaciones o despegues cinemáticos, disparando eventos lógicos estructurados al finalizar el recorrido para devolver el control dinámico al piloto inercial tradicional.
- **CinematicFlythrough.cs (Capa: Control de Cámaras Coreográficas):** Administra el recorrido y encuadre de cámaras independientes a lo largo de nodos de movimiento de diseño (waypoints) para capturar planos espectaculares del entorno. Opera mediante Lerp en lugar de MoveTowards, lo que provoca que la cámara nunca llegue exactamente de forma matemática al punto estricto, aunque cumple adecuadamente con su propósito estético y narrativo.
- **CameraDelay.cs (Capa: Gestión de Transición Cinemática / Cinemachine):** Actúa como un gestor intermedio encargado de coordinar los tiempos de retraso y las prioridades de peso dentro del sistema de cámaras inteligentes Cinemachine. Suaviza la transición de prioridades cuando el juego pasa de una secuencia coreográfica guiada por un spline cinemático (Dolly Track) al control manual de vuelo físico en tercera persona, evitando sacudidas bruscas en el FOV o la posición.
- **CameraController.cs:** Clase de script completamente vacía. No implementa ninguna lógica ni funcionalidad en la arquitectura actual del sistema.

3. Matriz de Diagnóstico de Problemas y Plan de Acción

Con el fin de mitigar la deuda técnica identificada en el mapa de dependencias actual, se establece la siguiente matriz de refactorización:

Componente afectado	Problema / Diagnóstico Detectado	Impacto en el Proyecto	Solución Arquitectónica / Plan de Acción
ShipProperties.cs	Uso de MonoBehaviour para persistir datos estáticos e inerciales del chasis de la nave.	Duplicidad de consumo de memoria por instancia y rigidez en la creación de variantes de naves.	Migrar a ScriptableObject. Separar los datos estáticos de configuración del ciclo de vida del objeto físico de Unity.
InputProvider.cs	Dependencia centralizada estricta del Legacy Input System.	Riesgo de incompatibilidad técnica si el proyecto es configurado para utilizar las nuevas APIs de Input de Unity.	Refactorizar el backend interno del script para dar soporte adaptativo o migrar al New Input System .
PlayerShipDriver.cs y OrbitalPlayerDriver.cs	Duplicación explícita de responsabilidades en la captura e implementación del driver del jugador humano.	Código redundante, acoplamiento innecesario con UnityEngine.Input heredado y complejidad en el mantenimiento.	Unificación de componentes. Eliminar PlayerShipDriver y consolidar la lectura de inputs limpios y Raycasting dentro del script único OrbitalPlayerDriver.cs.

Componente afectado	Problema / Diagnóstico Detectado	Impacto en el Proyecto	Solución Arquitectónica / Plan de Acción
OrbitalVisualizatio n.cs	Bug activo de excepción de referencia nula (NullReferenceExc eption).	Fallo crítico en el renderizado de órbitas físicas tridimensionales y esferas de influencia (SOI).	Depuración y Control de Nulos. Asegurar la correcta inicialización del componente LineRenderer y validar las referencias de datos de cuerpos celestes antes de ejecutar el render.
CameraController. cs	Script vacío (Código muerto / Dead Code).	Aumento innecesario de archivos huérfanos y confusión en el árbol jerárquico del desarrollo.	Eliminación del archivo. Remove por completo la clase del repositorio del proyecto.

4. Estructura de Carpetas Recomendada (Organización de Directorios)

Para materializar las mejoras arquitectónicas propuestas y asegurar que los módulos permanezcan completamente desacoplados y organizados de manera profesional, se debe aplicar la siguiente distribución de archivos en el directorio del proyecto:

```

Scripts/
├── Core/           ← Lógica fundamental de la nave y leyes del mundo físico
│   ├── Ship/
│   │   ├── OrbitalStarshipController.cs
│   │   ├── ShipInputState.cs
│   │   ├── IShipDriver.cs
│   │   └── ShipProperties.cs   ← (Planificado: migrar a ScriptableObject)

```

